Pontifícia Universidade Católica de Minas Gerais
Bacharelado em Engenharia de Software
Laboratório de Computação II
Prof.: Henrique Batista da Silva
Maratona de Programação 3 – 2,0 pontos. Entrega: **24/04/18 (SGA) às 20h40**.

Orientações:

- Os exercícios devem ser feitos em **linguagem Java**.
- A atividade deve ser feita em um único arquivo .java (extensão dos arquivos da linguagem Java) que deve ser entregue no SGA (**entregar no SGA apenas o arquivo .java**).
- Exercícios copiados receberão nota zero.
- **Não deixe a lista para a última hora**. Comece o quanto antes. Assim haverá tempo para esclarecer dúvidas com o professor e na monitoria.

Desafio de maratona de programação

Problema retirado do site do codeforces ("tradução" livre)

http://codeforces.com/problemset/problem/911/E

Explicação rápida:

Considere um vetor a como entrada, uma pilha s (inicialmente vazia) e um vetor b (inicialmente vazio).

Você pode executar as seguintes operações em a e s:

- Pegar o primeiro elemento de a, colocá-lo na pilha s e removê-lo de a (se a não está vazio);
- Pegar o topo de s, inserir no fim do vetor b e remover o elemento de s (se s não estiver vazio)

Estas operações podem ser realizadas em qualquer ordem. Observe que não é possível passar o elemento de a para b diretamente. Também não é possível passar elementos da pilha s de volta para o vetor a e nem do vetor b para pilha s ou vetor a.

Se for possível encontrar alguma forma de obter o vetor b ordenado de forma ascendente ao final do processo, o vetor a é chamado de stack-sortable.

Por exemplo, $[3, 1, 2]$ é stack-sortable, porque b será ordenado se executar as seguintes operações:

1. Remove 3 from *a* and push it into *s*;
2. Remove 1 from *a* and push it into *s*;
3. Remove 1 from *s* and append it to the end of *b*;
4. Remove 2 from *a* and push it into *s*;
5. Remove 2 from *s* and append it to the end of *b*;
6. Remove 3 from *s* and append it to the end of *b*.

Ao final de todas estas operações $b = [1, 2, 3]$, então [3, 1, 2] é stack-sortable. Veja que [2, 3, 1] não é stack-sortable.

Será fornecida k elementos de uma permutação de um vetor de n elementos. Você deve encontrar os $n - k$ elementos restantes do vetor. Testar quais são stack-sortable (se houver) e retornar o vetor lexicograficamente maximal (se houver mais de uma resposta).

Exemplo: 3 2 1 5 4 é lexicograficamente maior do que o vetor 3 2 1 4 5.

Input

The first line contains two integers n and k ($2 \le n \le 200000$, $1 \le k < n$) — the size of a desired permutation, and the number of elements you are given, respectively.

The second line contains k integers $p_1, p_2, ..., p_k$ ($1 \le p_i \le n$) — the first k elements of p. These integers are pairwise distinct.

Output

If it is possible to restore a stack-sortable permutation p of size n such that the first k elements of p are equal to elements given in the input, print lexicographically maximal such permutation.

Otherwise print -1.

Examples

Input
```
5 3
3 2 1
```
Output
```
3 2 1 5 4
```

Input
```
5 3
2 3 1
```
Output
```
-1
```

Input
```
5 1
3
```

Output
```
3 2 1 5 4
```

Input
```
5 2
3 4
```
Output
```
-1
```

Original.

E. Stack Sorting

Let's suppose you have an array *a*, a stack *s* (initially empty) and an array *b* (also initially empty).

You may perform the following operations until both *a* and *s* are empty:

- Take the first element of *a*, push it into *s* and remove it from *a* (if *a* is not empty);
- Take the top element from *s*, append it to the end of array *b* and remove it from *s* (if *s* is not empty).

You can perform these operations in arbitrary order.

If there exists a way to perform the operations such that array *b* is sorted in non-descending order in the end, then array *a* is called stack-sortable.

For example, [3, 1, 2] is stack-sortable, because *b* will be sorted if we perform the following operations:

7. Remove 3 from *a* and push it into *s*;
8. Remove 1 from *a* and push it into *s*;
9. Remove 1 from *s* and append it to the end of *b*;
10. Remove 2 from *a* and push it into *s*;
11. Remove 2 from *s* and append it to the end of *b*;
12. Remove 3 from *s* and append it to the end of *b*.

After all these operations $b = [1, 2, 3]$, so [3, 1, 2] is stack-sortable. [2, 3, 1] is not stack-sortable.

You are given *k* first elements of some permutation *p* of size *n* (recall that a permutation of size *n* is an array of size *n* where each integer from 1 to *n* occurs exactly once). You have to restore the remaining $n - k$ elements of this permutation so it is stack-sortable. If there are multiple answers, choose the answer such that *p* is lexicographically maximal (an array *q* is lexicographically greater than an array *p* iff there exists some integer *k* such that for every $i < k$ $q_i = p_i$, and $q_k > p_k$). You may not swap or change any of first *k* elements of the permutation.

Print the lexicographically maximal permutation p you can obtain.

If there exists no answer then output -1.

Input

The first line contains two integers n and k ($2 \le n \le 200000$, $1 \le k < n$) — the size of a desired permutation, and the number of elements you are given, respectively.

The second line contains k integers $p_1, p_2, ..., p_k$ ($1 \le p_i \le n$) — the first k elements of p. These integers are pairwise distinct.

Output

If it is possible to restore a stack-sortable permutation p of size n such that the first k elements of p are equal to elements given in the input, print lexicographically maximal such permutation.

Otherwise print -1.

Examples

Input
```
5 3
3 2 1
```
Output
```
3 2 1 5 4
```

Input
```
5 3
2 3 1
```
Output
```
-1
```

Input
```
5 1
3
```
Output
```
3 2 1 5 4
```

Input
```
5 2
3 4
```
Output
```
-1
```

Bom trabalho!